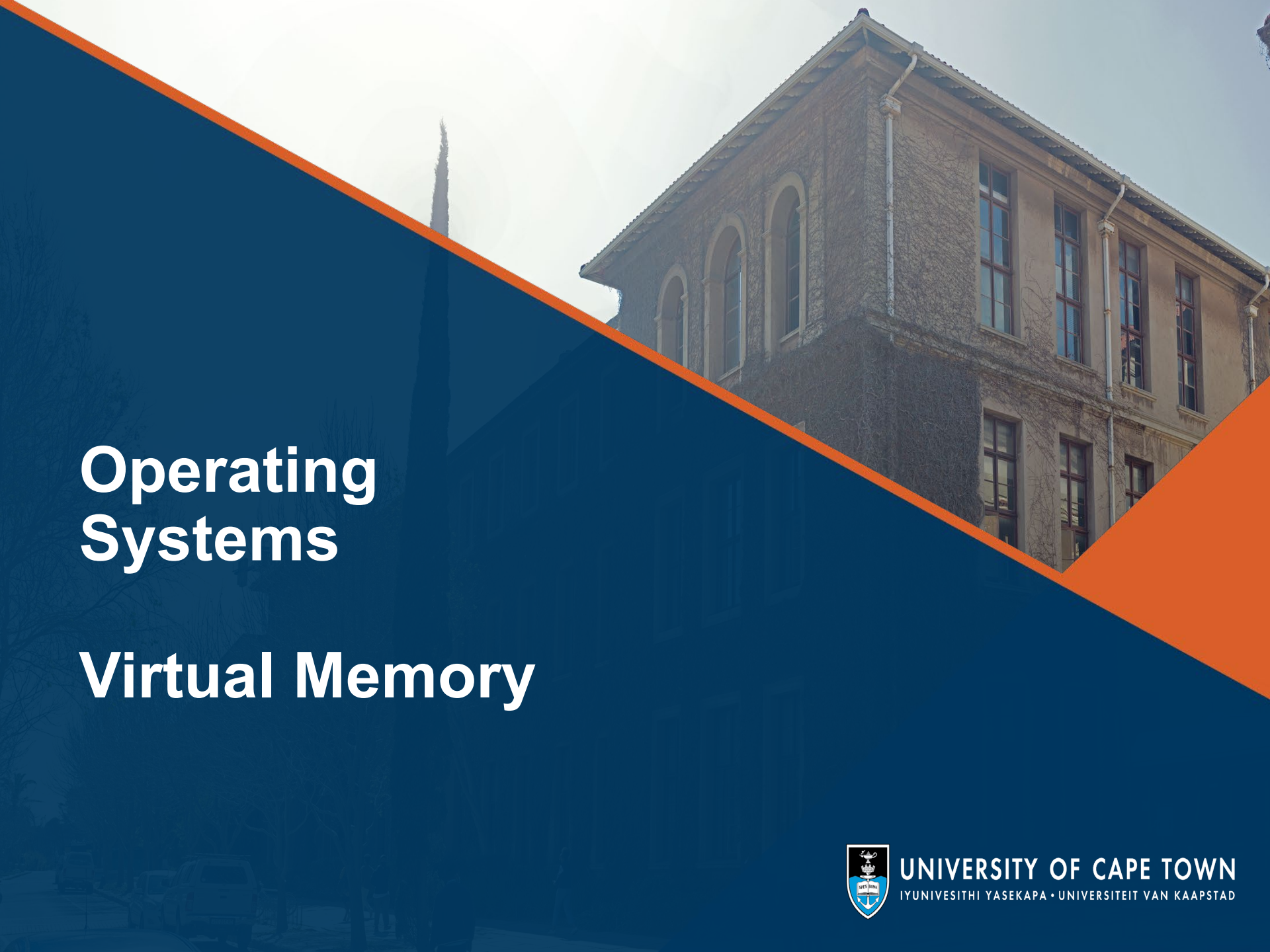


Operating Systems

Virtual Memory

UNIVERSITY OF CAPE TOWN
IYUNIVESITHI YASEKAPA • UNIVERSITEIT VAN KAAPSTAD

Virtual Memory: So Far…

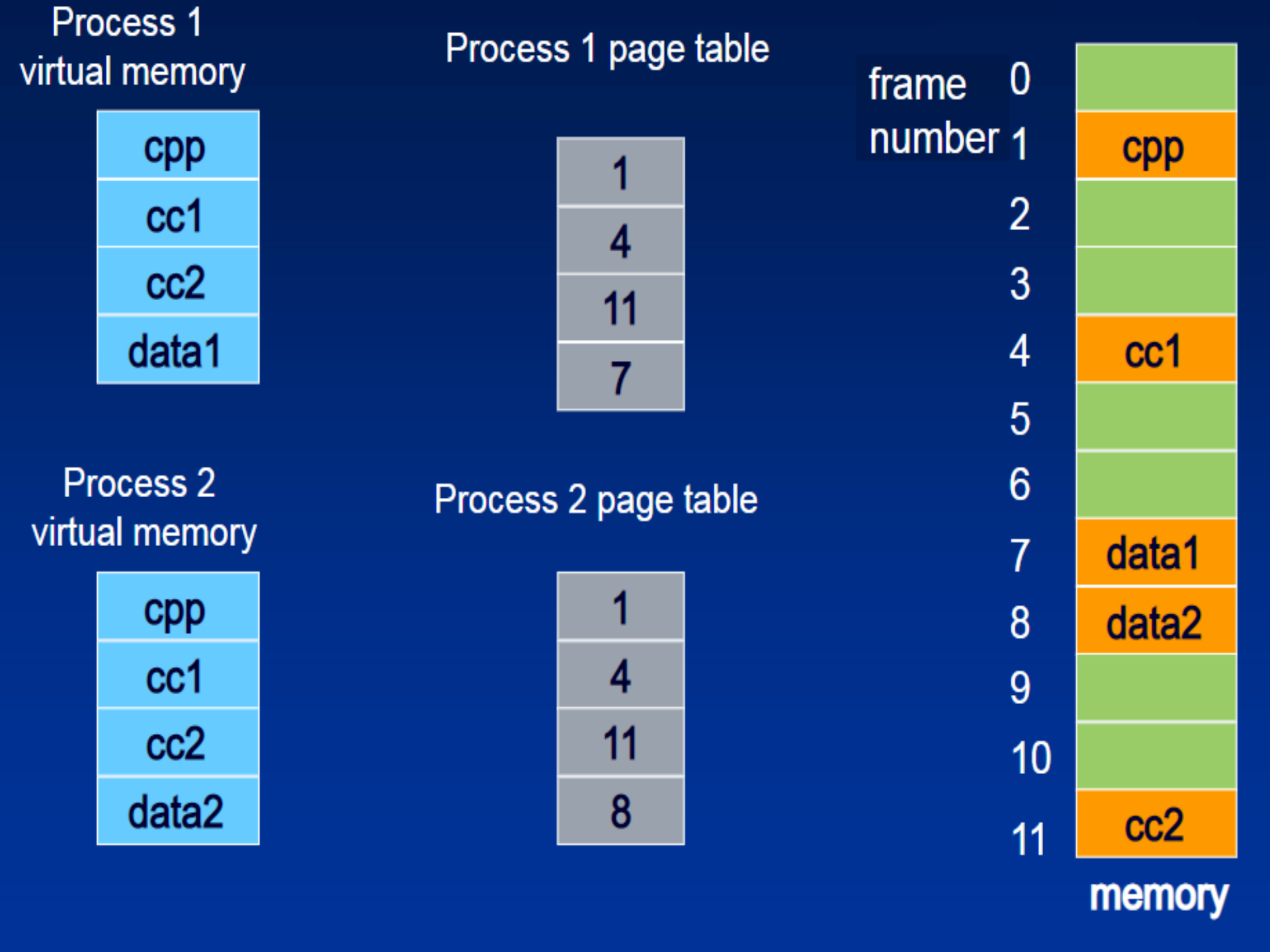
- Demand paging: Load pages into memory only when page faults occur.

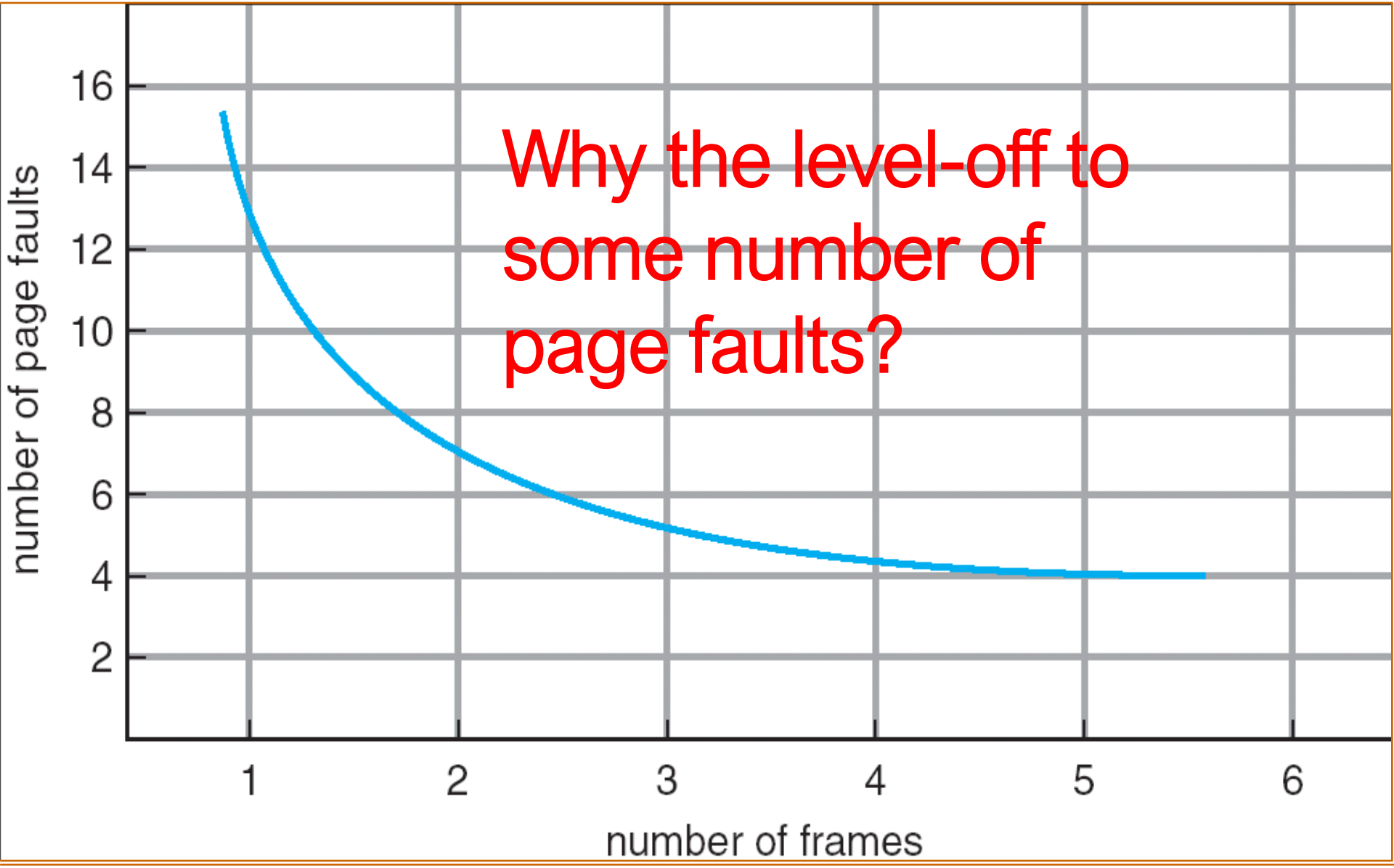
- **Previously – *Page replacement algorithms:*** What to do when there are more processes than can fit in memory.

- **Next – *Load control strategies*:** How many processes can fit in memory at one time.

Physical Frame Allocation

- How do we allocate memory among different processes?
- Does every process get the same fraction of memory? Different fractions?
- Should we completely swap some processes out of memory?


Process 1
virtual memory
cpp
cc1
cc2
data1
Process 1 page table
1
4
11
7
Process 2
virtual memory
cpp
cc1
cc2
data2
Process 2 page table
1
4
11
8
frame
number
0
1
2
3
4
5
6
7
8
9
10
11
cpp
cc1
data1
data2
cc2
memory

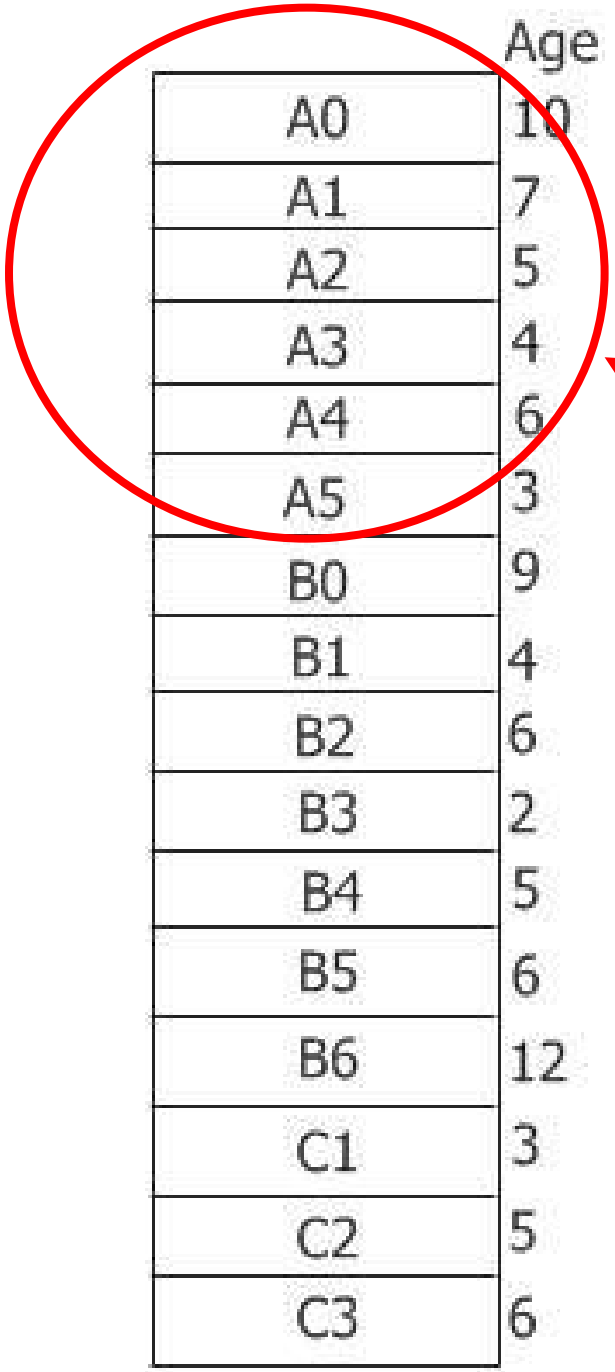

Why the level-off to some number of page faults?
number of page faults
16
14
12
10
8
6
4
2
1
2
3
4
5
6
number of frames

Where to Allocate Page Frames?

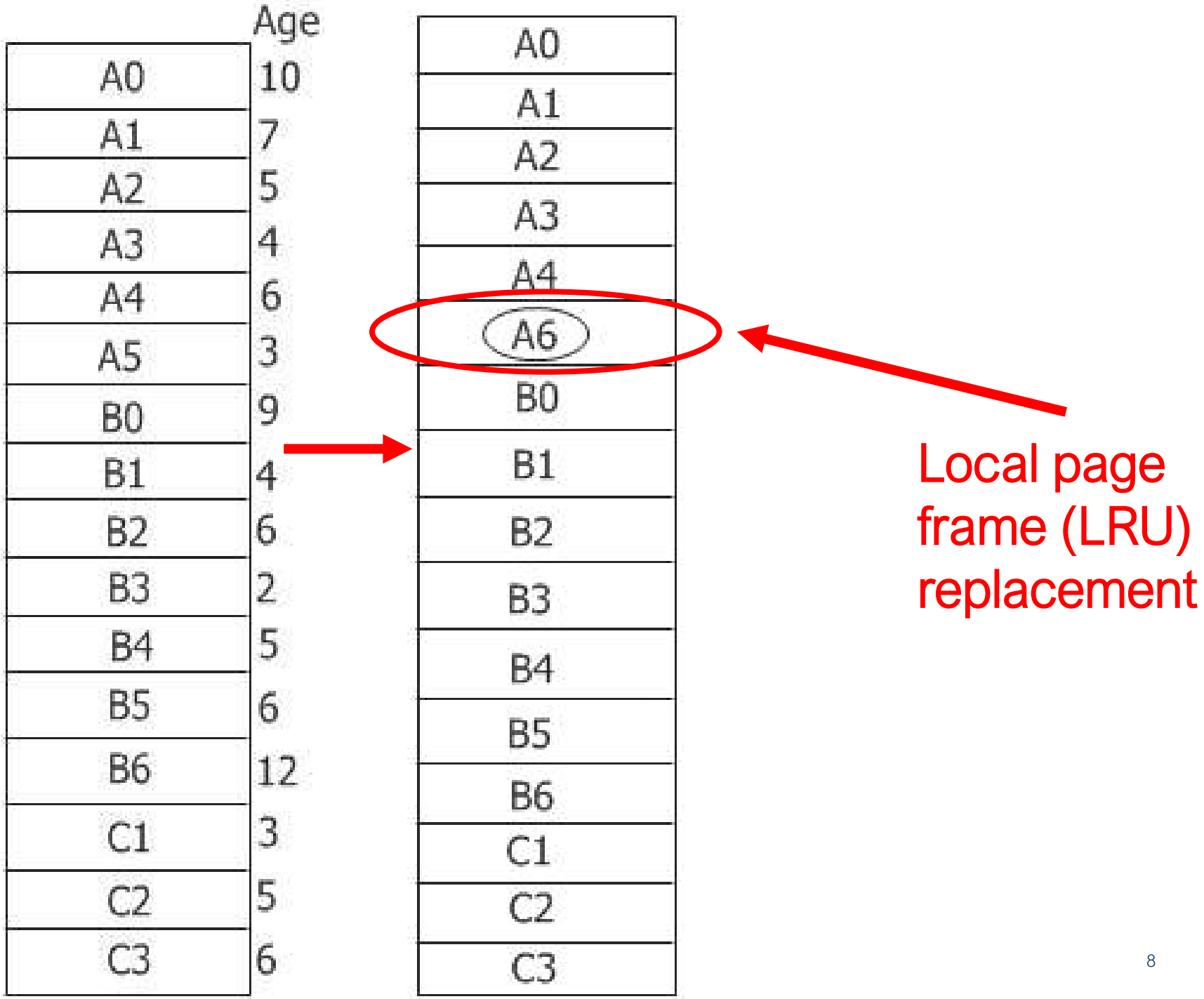
- All page replacement algorithms can be implemented using ***local*** or ***global*** replacement:

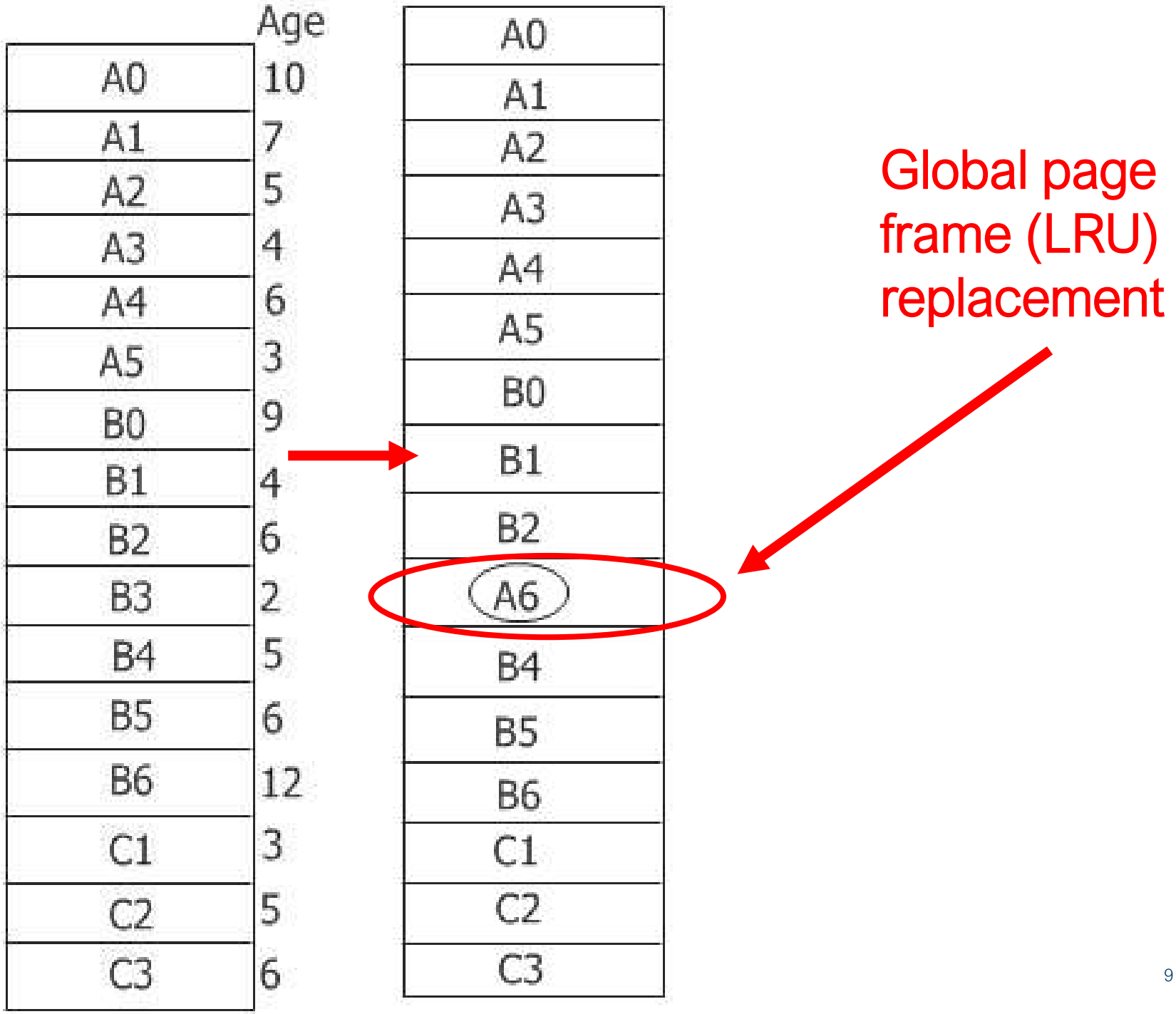
- **Local:** Process has a limit of pages it can use and *page against itself* (evicts its own pages).

- **Global:** *Victim page* chosen from all page frames (regardless of process).



Process A:
Page fault –
What frame to
replace?


Age
A0
10
A1
7
A2
5
A3
4
A4
6
A5
3
B0
9
B1
4
B2
6
B3
2
B4
5
B5
6
B6
12
C1
3
C2
5
C3
6
A0
A1
A2
A3
A4
A6
B0
B1
B2
B3
B4
B5
B6
C1
C2
C3
Local page frame (LRU) replacement


Age
A0
A1
A2
A3
A4
A5
B0
B1
B2
B3
B4
B5
B6
C1
C2
C3
10
7
5
4
6
3
9
4
6
2
5
6
12
3
5
6
A0
A1
A2
A3
A4
A5
B0
B1
B2
A6
B4
B5
B6
C1
C2
C3
Global page frame (LRU) replacement

Frame Allocation among Processes

- **Issues with local replacement?**

Types of Page Replacement

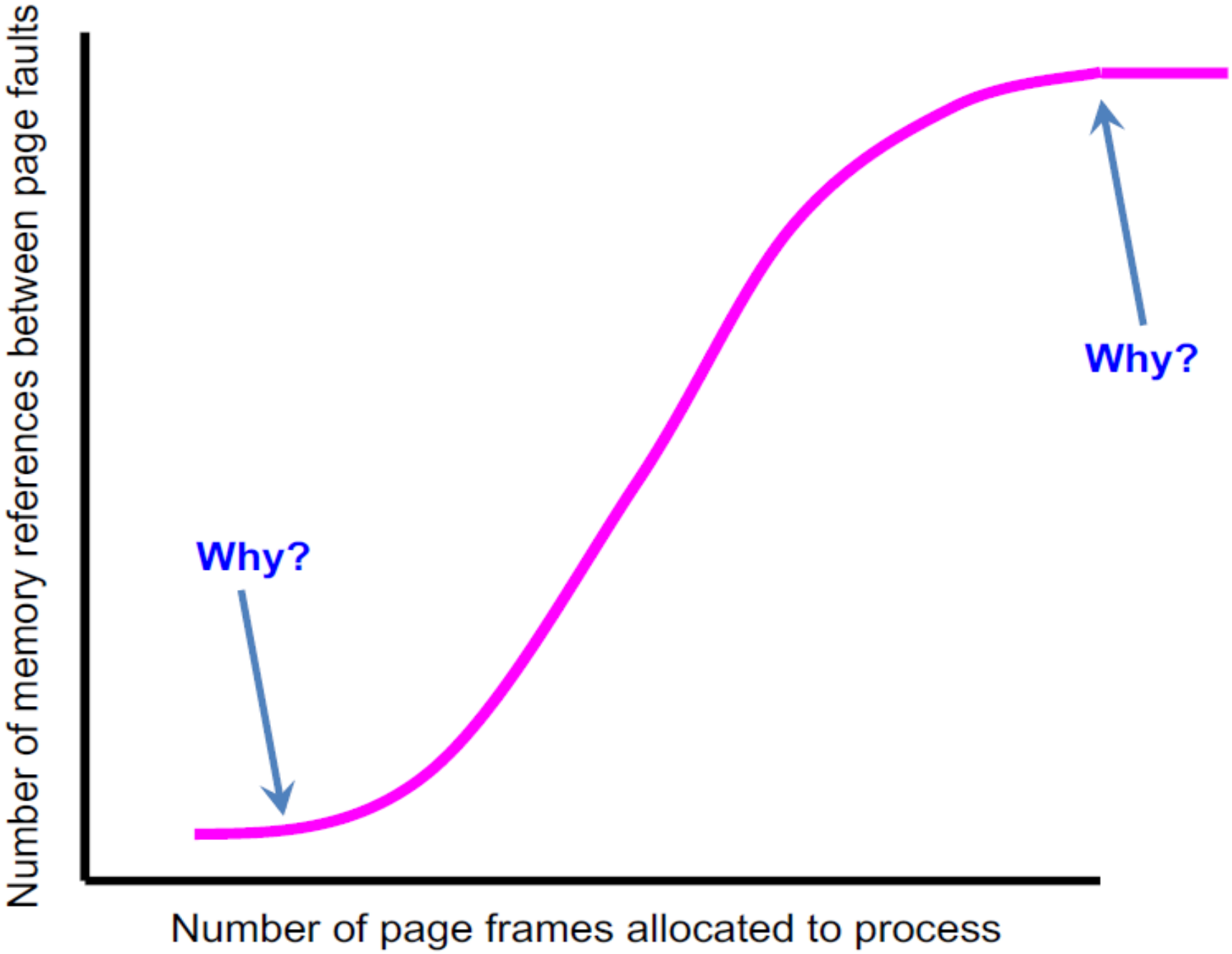
- **Local:** Process selects only its page frames:
  - More consistent per-process performance.
  - Potentially low per-process performance as memory requirements increase.

Frame Allocation among Processes

- Issues with global replacement?

Types of Page Replacement

- **Global:** Select from set of all process page frames – one process can take a frame from another process:
  - More efficient overall – but process run time can *vary greatly* – depends on other process demands.
  - *Memory hungry processes potentially spend all their time stealing frames from each other.*


Number of memory references between page faults
Number of page frames allocated to process
Why?
Why?

Frame Allocation Methods

- **Local replacement:** Processes allocated a fixed fraction of page frames.

- **Global replacement:** Page frames are dynamically allocated to N processes – so number of page frames varies per process.

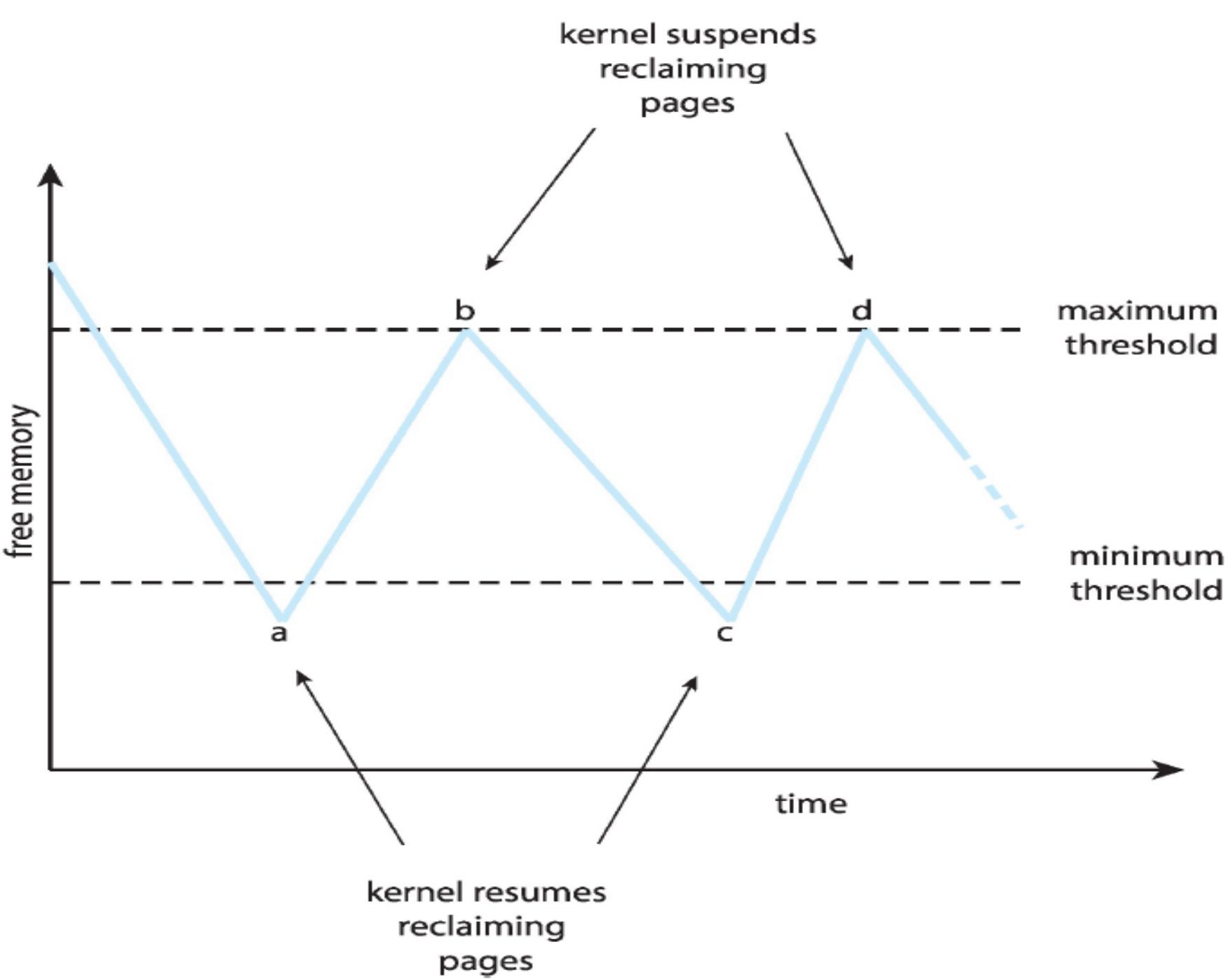
  - As memory demands grow and shrink each process can be allocated more or less page frames accordingly.

Reclaiming Pages

- A *global page replacement* policy.
- Use a ***free frame list***.
- Page replacement is triggered when number of free frames in the list falls below a ***certain threshold***.
- This strategy attempts to ensure ***there is always sufficient*** free memory to satisfy new requests.

What causes Page Replacement

- Compulsory Faults: Pages *currently not in physical (main) memory* – i.e. swapped out.
- Capacity Faults: *Not enough main memory* – Must somehow increase available memory size.
- Policy Faults: Pages were in memory – but kicked out because of *page replacement algorithm*.


kernel suspends
reclaiming
pages
b
d
maximum
threshold
free memory
minimum
threshold
a
c
time
kernel resumes
reclaiming
pages

Free Frames


7
8
9
10
11
12
13
14
15
16
17
Free frame list
7, 8, 10, 11,13, 16
frame number
Process creation
0
1
2
3
11
8
16
13
New process page table
Free frame list
7, 10
Page 1
Page 0
Page 3
Page 2
Before allocation
After allocation
physical memory

Fixed Page Frame Allocation

- Equal Allocation:
  - Every process gets same amount of memory:
  - **e.g.:** 100 frames, 5 processes: Each gets: 20 frames.

- Proportional allocation:
  - According to process size:

    s_i = size of process p_i and $S = \sum s_i$

    m = total number of frames

    a_i = (allocation for p_i) $= \frac{s_i}{S} \times m$

Linux(Ubuntu): v4.x → v22.x

- Maintains a list of free frames to assign to faulting processes.
- If there are less than *lots-free* (1/64 main memory) free frames, *page-out* (process) is started to try to free some more pages.
- *Page-replacement algorithm*: Clock algorithm.
- If *page-out* is unable to keep enough free pages, the kernel starts to swap out whole processes.

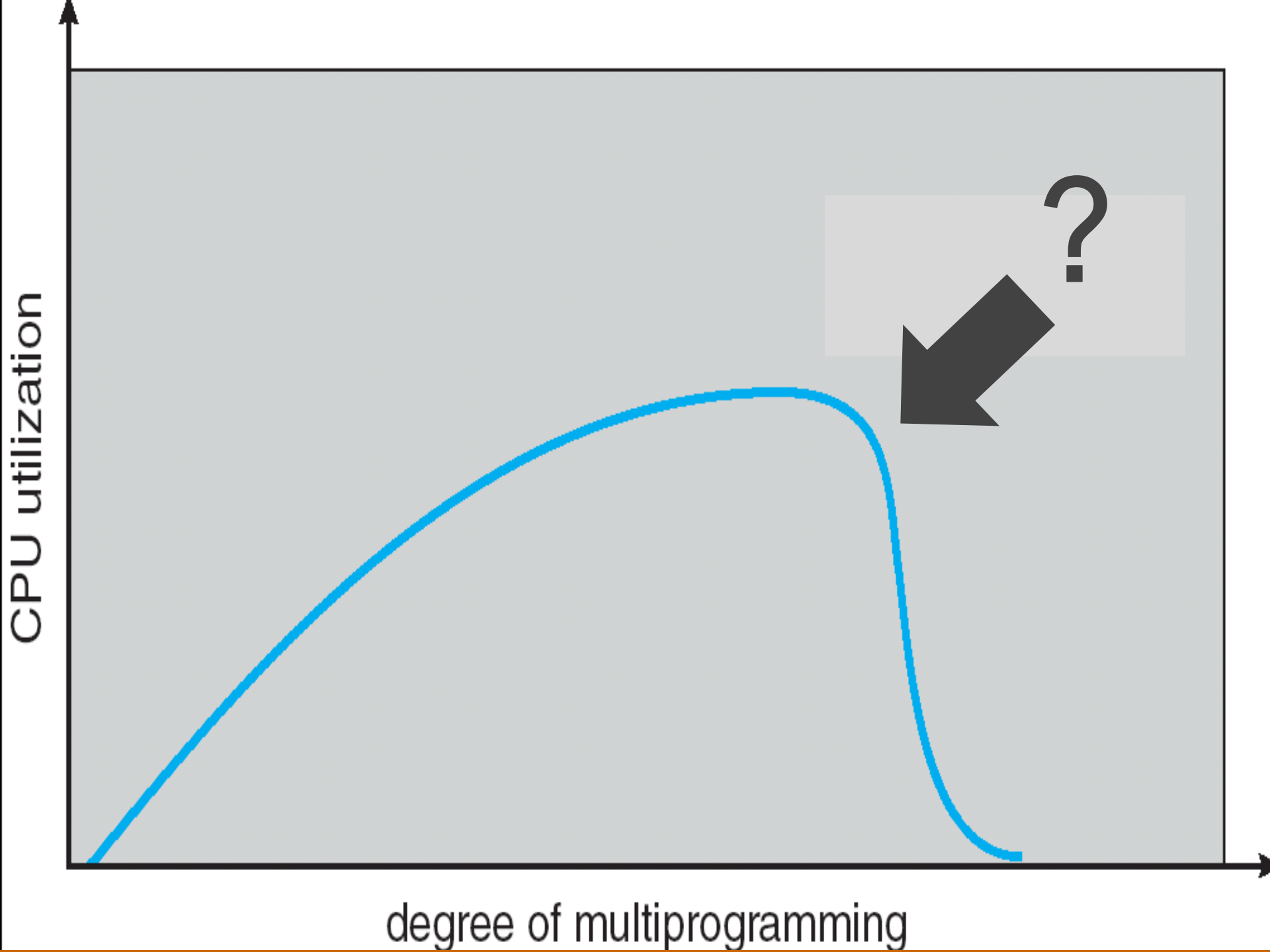
Priority Allocation

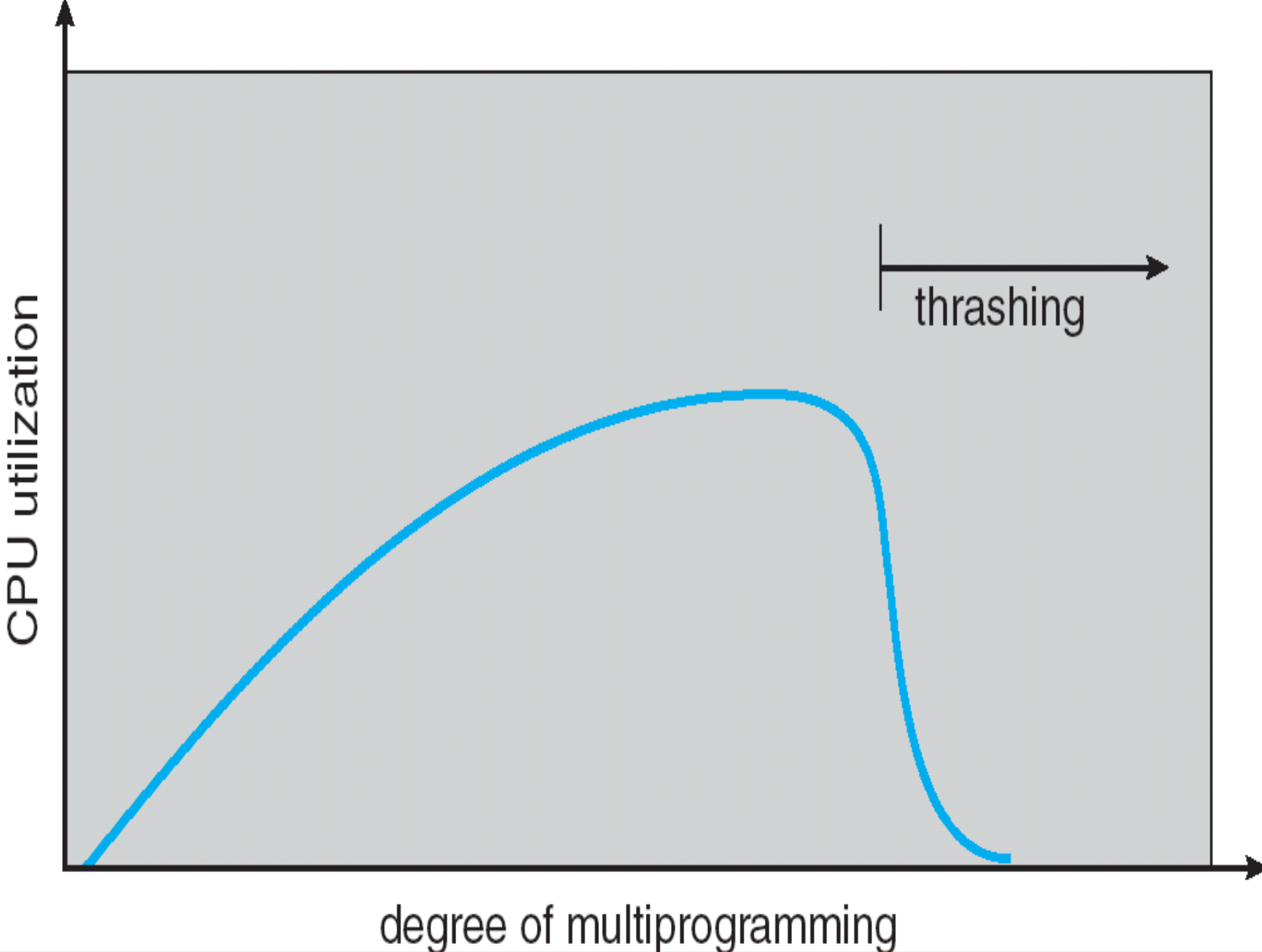
- Proportional scheme using ***priorities*** rather than *size*:
- Same type of computation as fixed allocation:
  - IF process p_i generates a page fault – THEN:
  - Select (for replacement) a frame from a process with lower priority number.
- *But – who gets what priority?*

Priority Allocation

- Priority of processes depends on:
  - OS Kernel process → Higher priority.
  - Time limit – e.g. longer running user process → lower priority.
  - Memory requirements → e.g. more frames used → lower priority.
  - Average I/O time – e.g. lower priority given to processes with slow I/O access.

Still more problems with page faults?


CPU utilization
?
degree of multiprogramming


CPU utilization
thrashing
degree of multiprogramming

What is Thrashing?

- Process spends all its time swapping pages in and out of physical memory.
- If a process does not have *enough* pages, then page-fault rate is very high – meaning:
  - Low CPU utilisation.
  - OS spends most of its time swapping to disk.
- *How to detect thrashing? And what to do?*

When does Thrashing occur?

- When the system spends most of its time servicing *page faults*, and little time doing useful work.

- **Enough memory – but:** Some bad page replacement algorithm, incompatible with program behaviour, is used.

- **Memory is over-committed:**
  - OS sees CPU is under-utilised so adds more processes.
  - But too many active processes only makes problem worse.

How to prevent Thrashing?

- Option 1:
  - ...?
- Option 2:
  - …?

How to prevent Thrashing?

- Option 1:
  - Limit thrashing effects by *using local or priority page replacement*.
  - One process starts thrashing but this does not affect others – i.e. cannot cause other process thrashing.
- Option 2:
  - …?

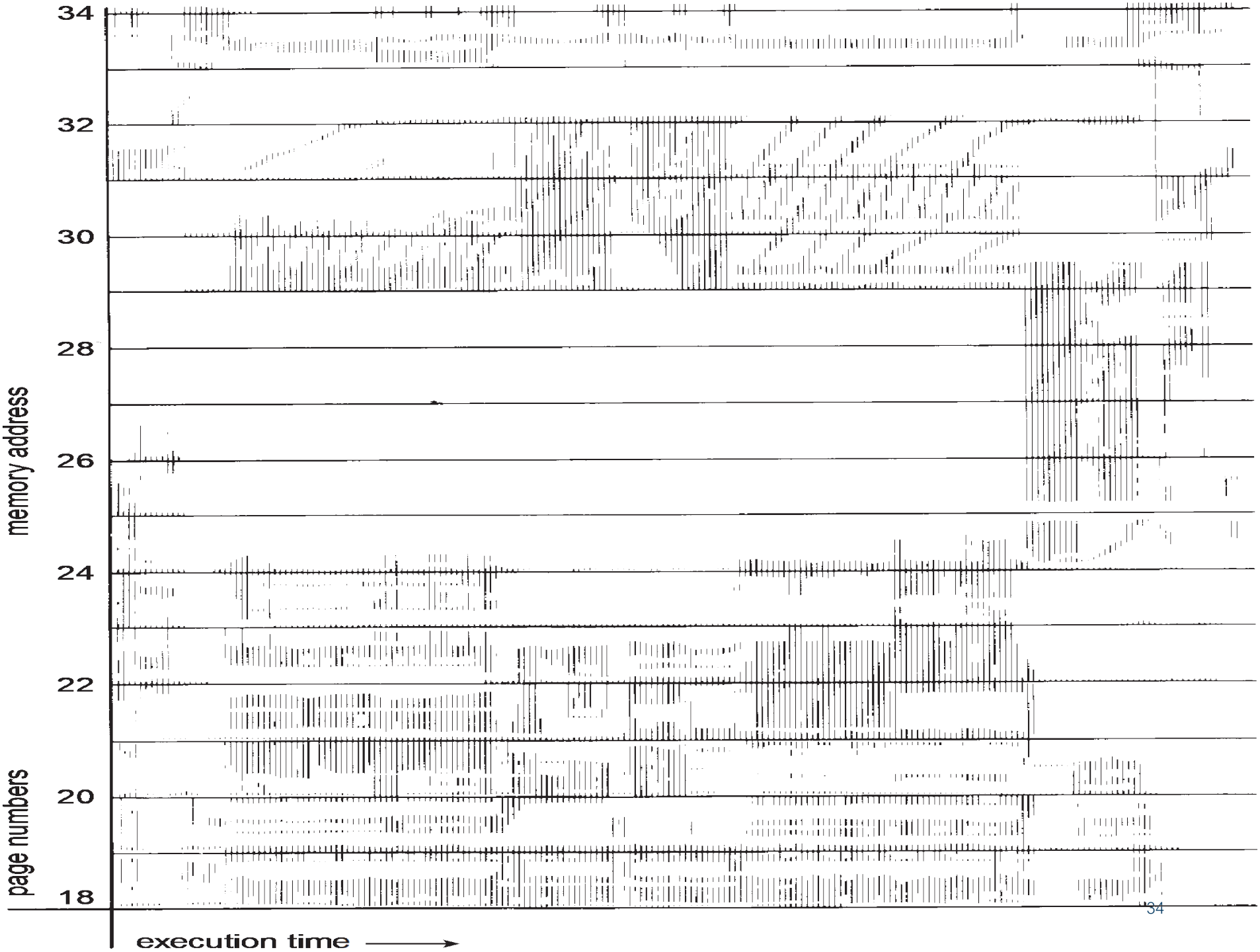
How to prevent Thrashing?

- Option 1:
  - Limit thrashing effects by *using local or priority page replacement*.
  - One process starts thrashing but this does not affect others – i.e. cannot cause other process thrashing.
- Option 2:
  - Only run processes – where *working sets* of these processes fit into X frames (Working set algorithm).

Working Set Model & Multi-Programming Load Control

Virtual Memory: Overview

- **Load control algorithms:** How many processes should be in memory at one time.


34
32
30
28
26
24
22
20
18
memory address
page numbers
execution time

Why does Virtual Memory Work?

- Virtual memory works because of ***locality***.
- **Locality of references:** references to memory by a process tends to be grouped.
- Only a few pieces of the process are used during a small time period (pages).
- Programs access a core set of active

Why does Virtual Memory Work?

- Programs with better (***temporal*** and ***spatial***) locality will have smaller working sets.

- If (***working set size*** < ***process memory size***) then: good performance for process after compulsory misses – ***else?***

Page Replacement Algorithms

- Random page replacement – Does not fit with program *principle of locality*.

- LRU (and derivative algorithms) – Works with *principle of locality* of programs.

Locality Example

- **Data:** Reference array elements in succession **(spatial)**.

```
sum = 0;
for (i = 0; i < n; i++)
   sum += a[i];
*v = sum;
```

- **Instructions:**
  - Reference instructions in sequence **(spatial)**.
  - Cycle through loop repeatedly **(temporal)**.

Program Structure

- **int[128, 128]** : *Assume page table has only one PTE – each array row **(i)** stored in one page*.

- **Program 1:**
  Page faults?

```
for (i = 0; i < 128; i++)
    for (j = 0; j < 128; j++)
        data[i,j] = 0;
```

Program Structure

- **int[128, 128]** : Assume page table has only one entry – *each array row **(i)** stored in one page*.

- **Program 1:** 128 page faults:

```
for (i = 0; i < 128; i++)
    for (j = 0; j < 128; j++)
        data[i,j] = 0;
```

Program Structure

- **int[128, 128]** : *Assume page table has only one PTE – each array row **(i)** stored in one page*.

- **Program 2:** Page faults?

```
for (j = 0; j <128; j++)
    for (i = 0; i < 128; i++)
        data[i,j] = 0;
```

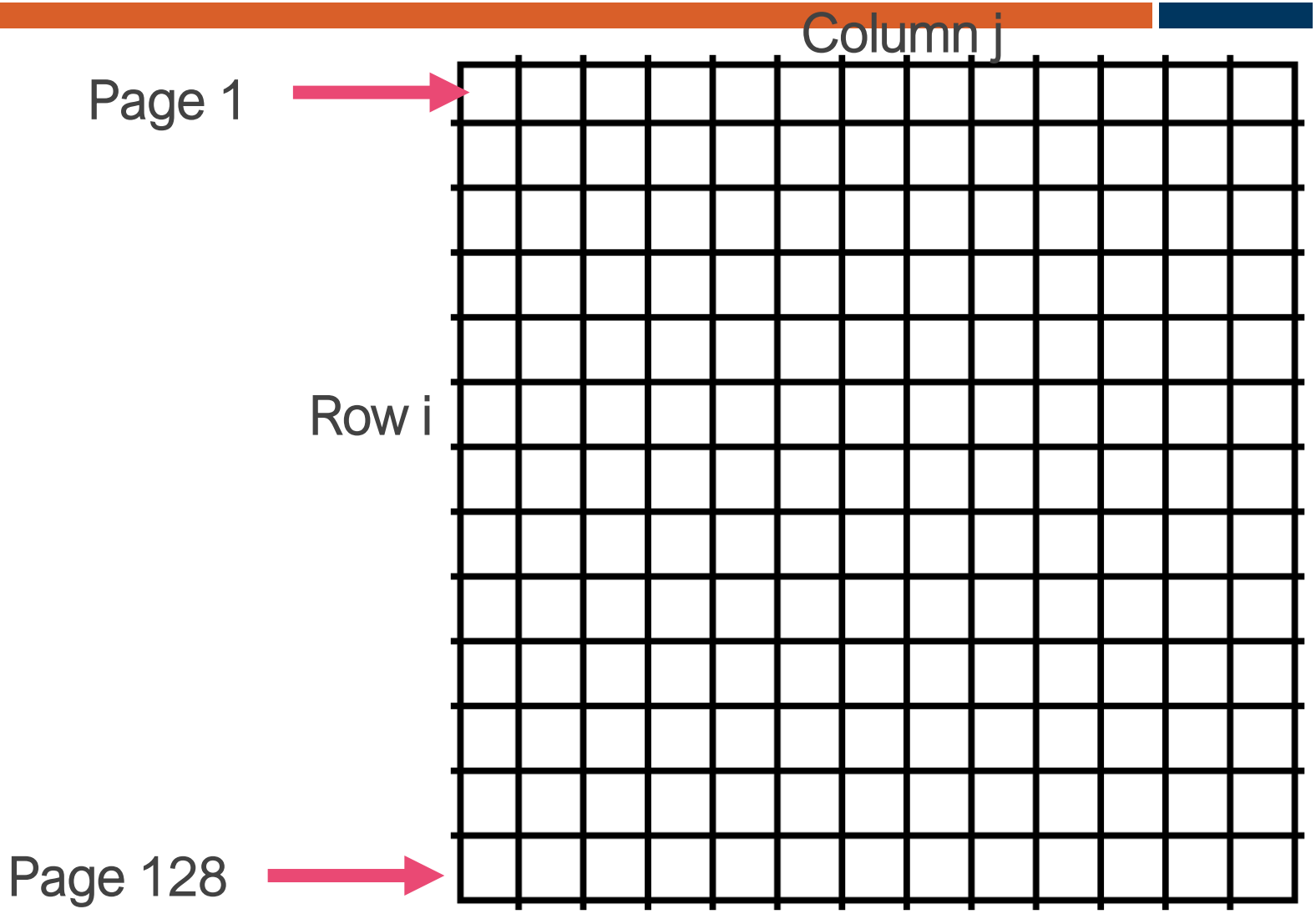
Program Structure

- **int[128, 128]** : Assume page table has only one entry – *each array row **(i)** stored in one page*.

- **Program 2:** 128x128 = 16,384 page faults:

```
for (j = 0; j <128; j++)
      for (i = 0; i < 128; i++)
            data[i,j] = 0;
```

Program Structure



Program Structure: Working Set

- **Working Set:** Sets of pages accessed in a given time period (**w**) – Defines minimum number of pages needed for process to *behave well* (i.e. minimal page faulting).

Use Locality: Working Set Model

- **Working-Set Model for Page Replacement:**
  - Assume recently referenced pages are likely to be referenced again soon…
  - ... and *only* keep those pages recently referenced in memory (i.e. ***process working set***).
  - Process executes *only if its working set fits into memory*.

Working Set Algorithm

- Estimate: **|WS(0, w)|** for each process:
  - Process starts only if can allocate it enough frames.
  - Use a *local page replacement* algorithm.
  - Track each process's working set size.
  - Re-allocate pages within processes dynamically.
  - *How do we calculate WS?*

Program Memory: Working Set

- **Working Set:** Sets of pages accessed in a given time period (**w**) – Defines minimum number of pages needed for process to *behave well* (i.e. minimal page faulting).

Working Set Definition

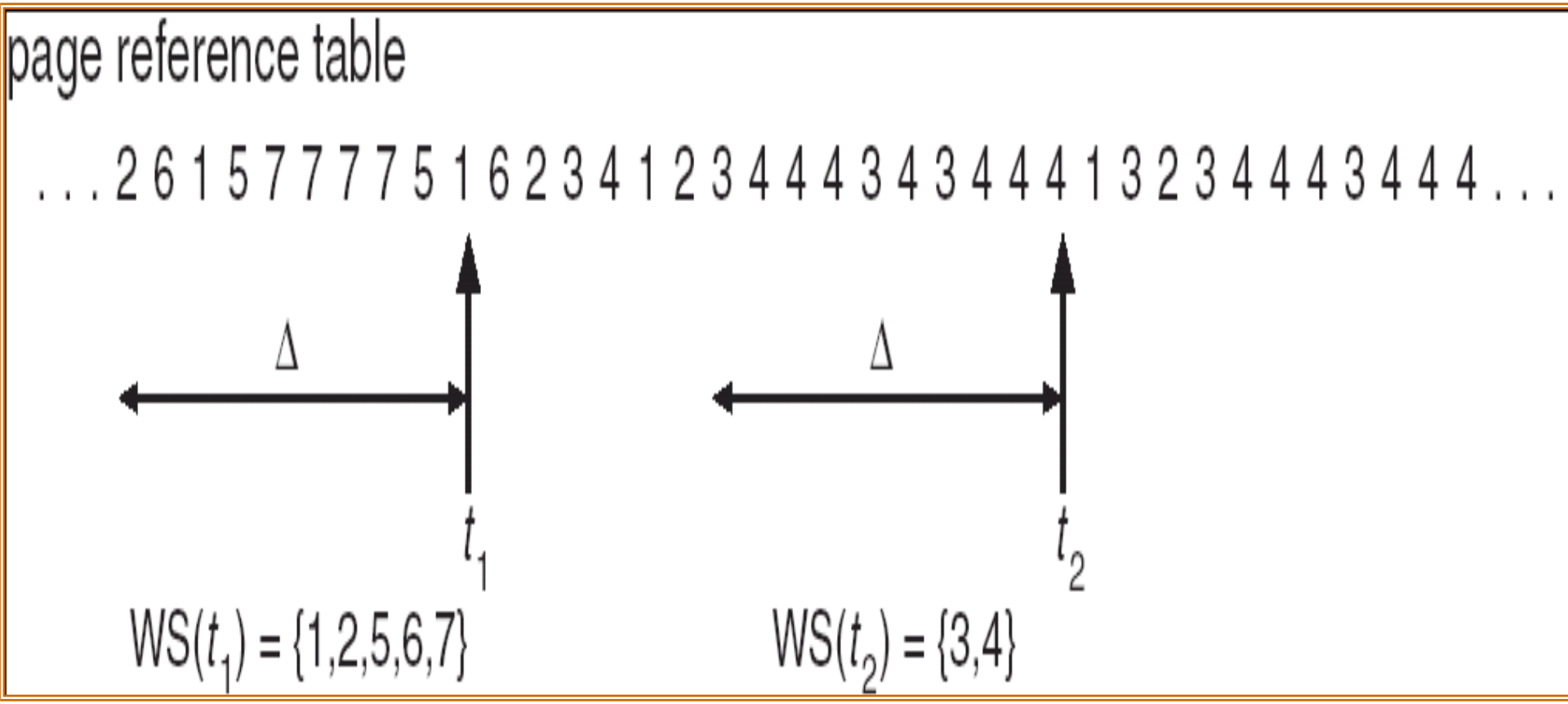
- Set of pages process currently *needs* :

WS(t, w) = { **P** pages, where **P** was referenced in time interval: **(t, t - w)** } ;

  - **t:** time ;
  - **w:** working set window (measured in page references) ;

- *Page is in **WS** only if referenced in last **w** references*.

Working Set Example



Working Set Size

- *Working Set* size: **|WS(t, w)|**:
  - *Changes with program locality*.
- Low temporal and spatial locality:
  - *…?*
- Longer time of low *temporal & spatial* locality:
  - *…?*

Working Set Size

- *Working Set* size: **|WS(t, w)|**:
  - *Changes with program locality*.
- Low temporal and spatial locality:
  - *More different pages referenced*.
- Longer time of low *temporal & spatial* locality:
  - *Working set size is larger*.

Back to Page Fault Frequency

- Track (**PFF**) in each process:
  - Record time (***t-last-PF***) of last page fault versus current time (***t-current***).
  - Look at: (**w**): working set window size.
  - IF ***Page Refs (t-current – t-last-PF) > w*** THEN …?
  - IF ***Page Refs (t-current – t-last-PF) ≤ w*** THEN …?

Working Set Page Replacement

IF Page Refs: ((**t-current) – (t-last-PF)**) **> *w*** **THEN**

remove from memory all pages not referenced

in period: **[t-last, t-current]**;

IF Page Refs: ((**t-current) – (t-last-PF)**) **≤ *w*** **THEN**

add faulting page to main memory;

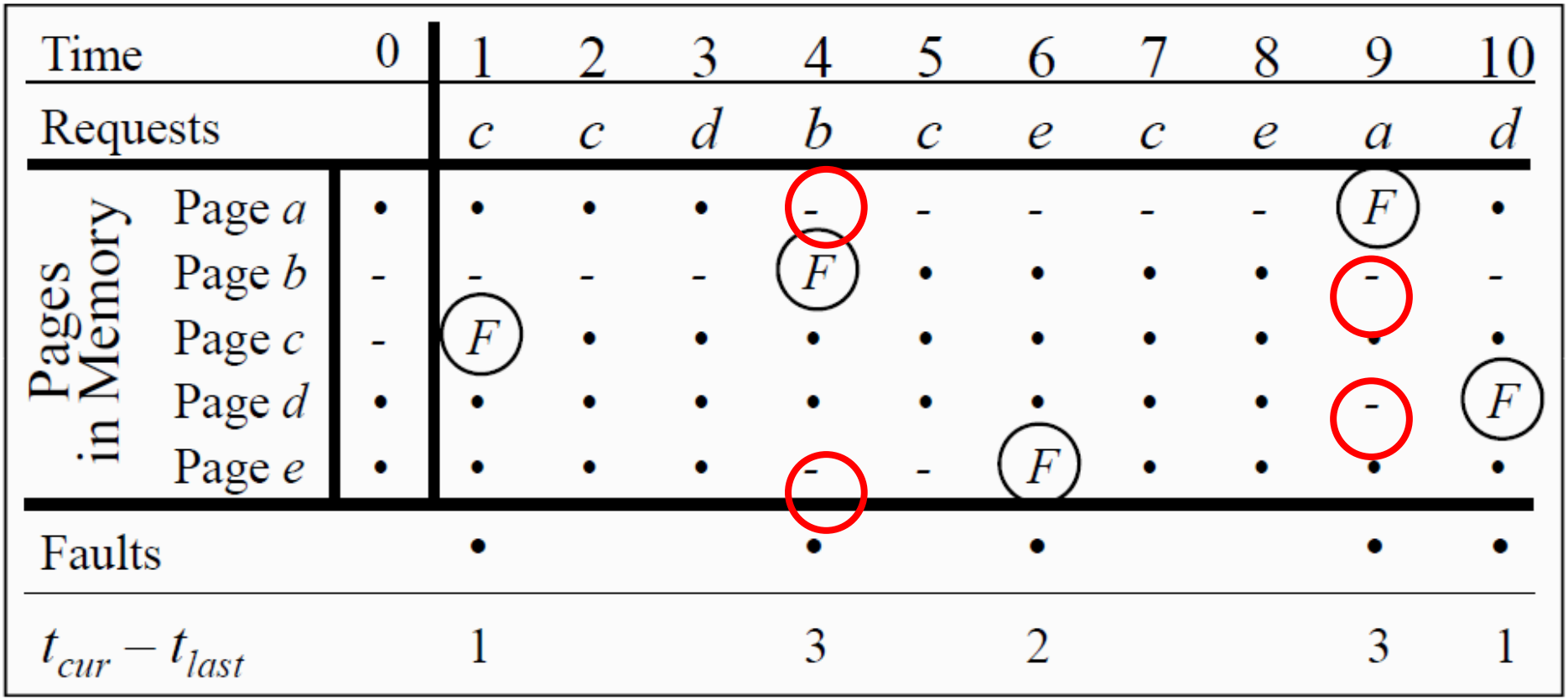
Example: Window size (w) = 2

- **IF** [t-current – t-last] **>** 2 **THEN** remove from memory all pages not referenced in: [t-last, t-current];
- **IF** [t-current – t-last] **<** 2 **THEN** add faulting page to

Time		0	1	2	3	4	5	6	7	8	9	10
Requests			*c*	*c*	*d*	*b*	*c*	*e*	*c*	*e*	*a*	*d*
Pages in Memory	Page *a*	•	•	•	•	-	-	-	-	-	F	•
	Page *b*	-	-	-	-	F	•	•	•	•	-	-
	Page *c*	-	F	•	•	•	•	•	•	•	•	•
	Page *d*	•	•	•	•	•	•	•	•	•	-	F
	Page *e*	•	•	•	•	-	-	F	•	•	•	•
Faults			•			•		•			•	•
$t_{cur} - t_{last}$			1			3		2			3	1

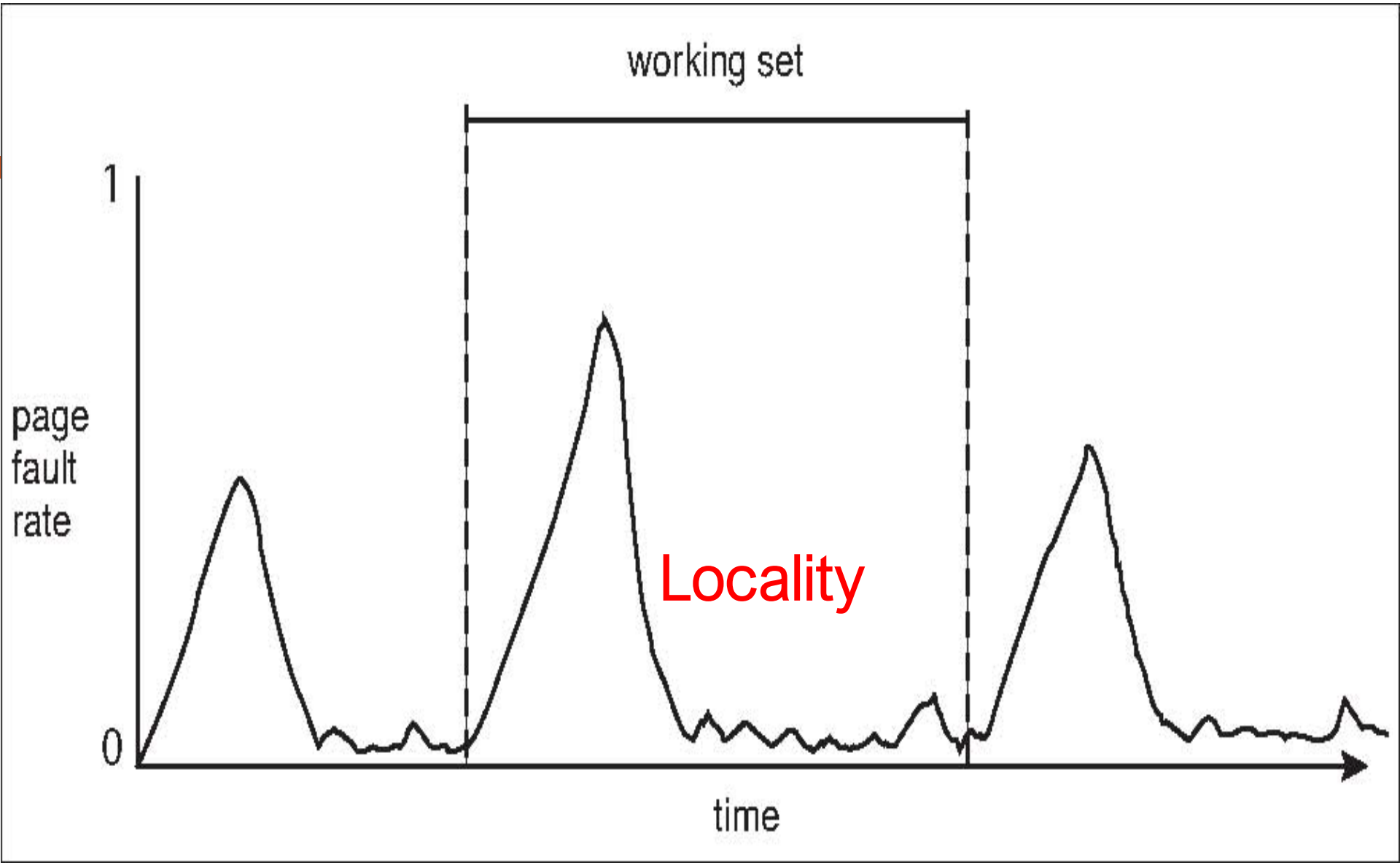
Example: Window size (w) = 2

- **IF** [t-current **–** t-last] **>** 2 **THEN** remove from memory all pages not referenced in: [t-last, t-current];



How to Estimate Working Set?

- How many pages does a process need?
  - i.e.: what is its working set size?
  - Informally – working set is all pages a process is currently using.
  - Formally – it is the set of all pages that a process referenced in the past **w** seconds.
  - But – what should **w** be…?


working set
1
page
fault
rate
Locality
0
time

How to Estimate Working Set Size?

- How to select **w** ? – consider:
  - 1 page fault = 10 milliseconds.
  - 10 milliseconds ~ 2 million instructions.
  - So: **w** must be much larger than 10ms (2 million instructions).
  - *What happens if **w** is too small or too large?*

Working Set: What Size fits Best?

- **WSi** : Working set of process; **Pi** : *Total set of pages referenced in the most recent* **w**.

  w ≡ *working-set window* ≡ fixed number of page references.

  D ≡ *Total page frame demand* of **WSi**.

  m ≡ *Memory allocated* to **Pi**.

- If **w** *too small* or *too large* ⇒ … ?

- If **D > m** ⇒ … ?

Working Set Size

- If **w** is too small:
  - ...
- If **w** is too large:
  - ...

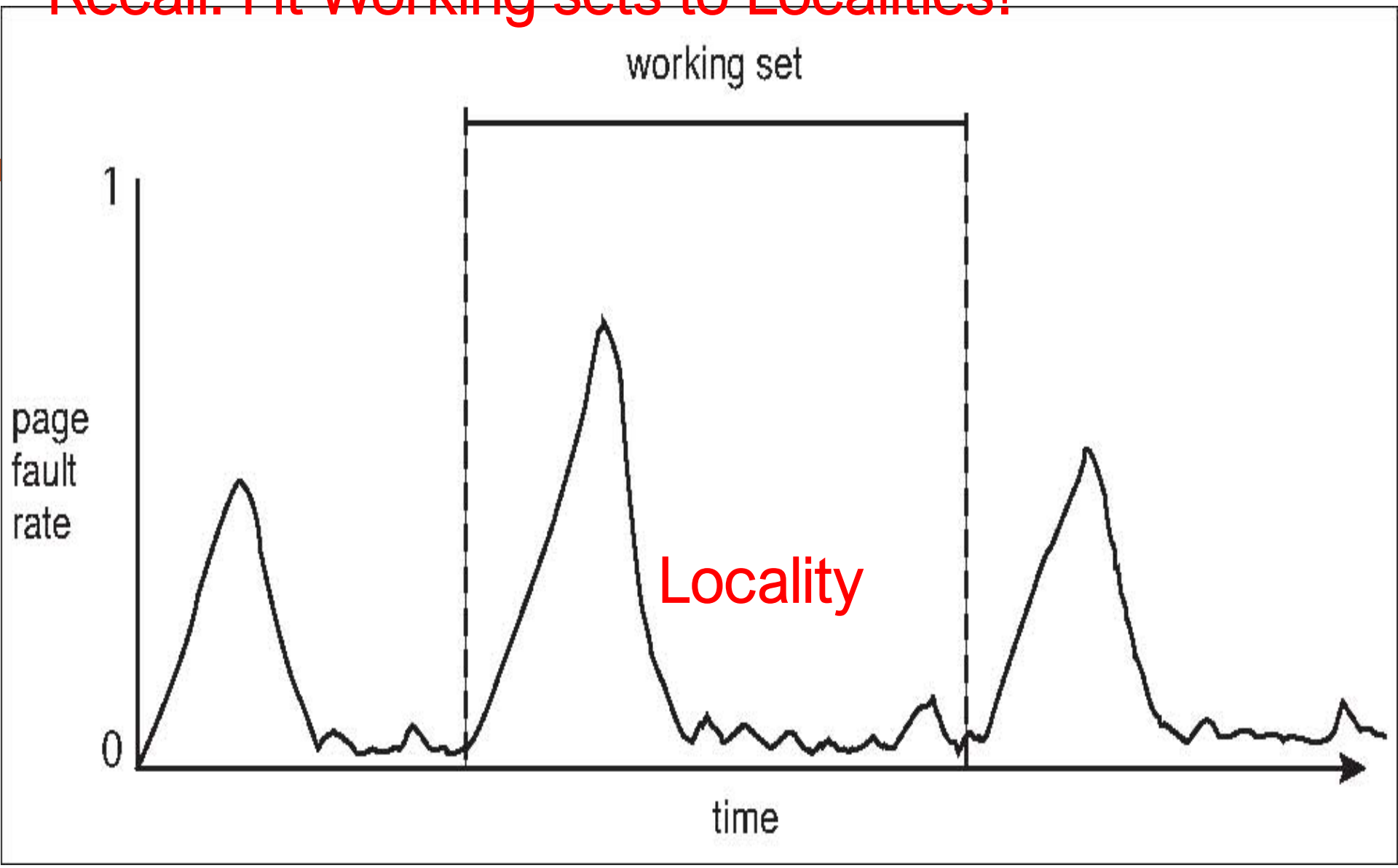
Working Set Size

- If **w** is too small:
  - **WSi** will not encompass entire locality – slows down process (lots of page faults and swapping).
- If **w** is too large:
  - ...

Working Set Size

- If **w** is too large:
  - **WSi** will encompass several localities – too many page references for this process – inefficient memory use.
  - If **w** $\Rightarrow \infty \Rightarrow$ **WSi** will encompass entire program.
- If **D > m** $\Rightarrow$ …

Recall: Fit Working sets to Localities!



Working Set Size

- If **w** is too large:
  - **WSi** will encompass several localities – too many page references for this process – slows down other processes.
  - If **w** $\Rightarrow \infty \Rightarrow$ **WSi** will encompass entire program.
- If **D > m** $\Rightarrow$ *…?*

Working Sets: Overview

- **Working Set:** Pages recently used by process.
  - **D ≡ |WSi|** total demand for pages; **m: Pi** memory.
  - **D > m** ⇒ Thrashing (global replacement) / *Slow (local replacement)*
- IF Pi **D > m** : Process does not fit into memory (X frames allocated for this process).
- IF Pi **D > m** THEN swap out (suspend) process Pi.

Summary: Working Set Model

- Process ***behaves well*** if its working set fits into memory.
- ***Badly behaved*** processes are suspended (swapped out).
- **Runtime:** A process migrates from one locality to another.
- **Working set algorithm:** Assumes recently referenced pages likely to be referenced again soon – *only* keep pages recently referenced in memory.
- **Working set algorithm:** Is a type of load control model!